

A Real Time Project Report on
FAULT DETECTION IN COMBINATIONAL
CIRCUITS

BACHELOR OF TECHNOLOGY

In

ELECTRONICS AND COMMUNICATION ENGINEERING

A Real Time Project Report Submitted By

B. HARSHA VARDHAN (23011Po410)

N. POOJITH (23011Po425)

A. MOKSHA (23011Po440)

CH. PRATHIBHA (23011Po451)

Under the esteemed guidance of

DR. A. RAJANI

Professor in ECE Department



Department Of Electronics and Communication Engineering
Jawaharlal Nehru Technological University Hyderabad
University College Of Engineering , Science & Technology
Hyderabad .

(Kukatpally -Hyderabad-500085)

2024-2025

Department of Electronics and Communication Engineering
Jawaharlal Nehru Technological University Hyderabad
University College Of Engineering , Science & Technology
Hyderabad .



CERTIFICATE BY THE SUPERVISOR

This is to certify that the project report entitled “**FAULT DETECTION IN COMBINATIONAL CIRCUITS**” being submitted by B. Harsha vardhan (23011Po410), N. Poojith (23011Po425), A. Moksha (23011Po440), CH. Prathibha (23011Po451) in partial fulfillment of the requirements for the award of degree in Bachelor of Technology in Electronics and Communication Engineering at the Jawaharlal Nehru Technological University during the academic year, 2024-2025 is a bonafide work carried out under my guidance and supervision. The results have been verified and found to be satisfied.

Supervisor:

Dr. A. Rajani

Professor in ECE Department.

**Department Of Electronics and Communication Engineering
Jawaharlal Nehru Technological University Hyderabad
University College Of Engineering , Science & Technology
Hyderabad .**



CERTIFICATE BY THE HEAD OF THE DEPARTMENT

This is to certify that the project report entitled “**FAULT DETECTION IN COMBINATIONAL CIRCUITS**” being submitted by B.Harshavardhan(23011Po440),N.Poojith(23011Po425),A.Moksha(23011Po440),C H.Prathibha(23011Po451)in partial fulfillment of the requirements for the award of degree in Bachelor of Technology in Electronics and Communication Engineering at the Jawaharlal Nehru Technological University during the academic year, 2024-2025 is a bonafide work carried out by them.

Head of Department
Dr. T. Madhavi Kumari
Professor and Head
Department of ECE
JNTUHCEH.

**Department Of Electronics and Communication Engineering
Jawaharlal Nehru Technological University Hyderabad
University College Of Engineering , Science & Technology
Hyderabad .**



DECLARATION OF THE CANDIDATES

We hereby declare The Real Time Project entitled “**FAULT DETECTION IN COMBINATIONAL CIRCUITS** ” is a bonafide record work done and submitted under the esteemed guidance of **Dr. A. RAJANI**, Professor, Department of ECE, JNTUH CEH, in partial fulfillment of the requirements for Real time project in Electronics and Communication Engineering at the Jawaharlal Nehru Technological University during the academic year 2024-25 is a bonafide work carried out by us and the results kept in the Real time project has not been reproduced. The results have not been submitted to any other institute or university for the award of a degree or diploma.

**B. HARSHA VARDHAN (23011Po410)
N. POOJITH (23011Po425)
A. MOKSHA (23011Po440)
CH. PRATHIBHA (23011Po451)**

ACKNOWLEDGEMENT

This project titled “**FAULT DETECTION IN COMBINATIONAL CIRCUITS** ” was carried out by us. We are grateful to Prof. Dr. A. Rajani Professor of Electronics and Communication Engineering, JNTU Hyderabad University College of Engineering, Science and Technology Hyderabad, for their guidance while pursuing this project.

We take this precious opportunity to acknowledge our internal project guide Prof. Dr. A. Rajani , Professor of Electronics and Communication Engineering, JNTUH University College of Engineering, Science and Technology Hyderabad for her timely advice, effective guidance, and encouragement throughout the completion of our real time project work.

We also owe a deep respect of gratitude to our parents and friends for their cheerful encouragement and valuable suggestions, without whom this work would not have been completed in this stipulated time.

We would like to articulate our heartfelt gratitude to the authorities of JNTUH for their help throughout our project work. A few lines of acknowledgment do not fully express our gratitude and appreciation for all those who guided and supported us throughout this project. Lastly, we acknowledge the help received from many journals and websites. Finally, we thank one and all who helped us directly or indirectly throughout our project work.

B. HARSHA VARDHAN (23011Po410)
N. POOJITH (23011Po425)
A. MOKSHA (23011Po440)
CH. PRATHIBHA (23011Po451)

	INDEX	
CONTENTS		Page Number
ABSTRACT		07
CHAPTER -1		08-10
Introduction to Fault Detection in Combinational Circuits		08
Merits and Demerits		09
Aim		10
Objectives		10
Applications		10
CHAPTER-2		11-19
Basic Components		11
CHAPTER-3		20-21
Working principle		20
CHAPTER-4		22-38
Fault Detection and Circuit Analysis		22
CHAPTER-5		39-42
Conclusion		39
Future Scope		39
REFERENCES		41

ABSTRACT

Combinational circuits are the fundamental building blocks of digital electronics, designed to produce specific outputs based solely on current input values, without memory elements. Common combinational circuits include adders, which perform binary addition; subtractors, which handle binary subtraction; encoders, which convert multiple input lines into a binary code; and decoders, which perform the inverse operation of encoders by decoding binary inputs into one active output line. These circuits are widely used in arithmetic logic units, control units, and data routing in digital systems.

Despite their simplicity, combinational circuits are prone to various faults that can compromise system reliability. Common fault types include stuck-at faults, where a line is fixed at logic '0' or '1'; bridging faults, where two signal lines are shorted; and open faults, where a line is disconnected. These faults can occur due to manufacturing defects, aging, or radiation effects, and may cause incorrect output values even when the input is correct. Early detection of such faults is essential for fault-tolerant and high-reliability digital systems.

Detecting these faults in real-time is crucial for ensuring system reliability. Traditional fault detection techniques, such as parity checking, duplication with comparison, often involve additional hardware, increasing power consumption, complexity, and area overhead. To address these challenges, a Finite State Machine (FSM)-based fault detection approach provides an efficient solution by continuously monitoring the adder's behavior and identifying errors with minimal hardware overhead.

It detects single stuck-at faults (SA0 and SA1) in combinational logic circuits using a Finite State Machine (FSM)-based approach. The method involves generating a fault-free and faulty truth table, identifying test vectors that expose faults, and optimizing them for full fault coverage. An FSM is then designed using counters to sequentially apply these test vectors, and any output deviation from the expected results indicates a fault in the circuit.

CHAPTER-1

INTRODUCTION

In digital electronics, combinational circuits play a crucial role in processing data through logic operations. These circuits are designed so that their output depends solely on the current inputs, without any memory or feedback from past inputs. They are built using logic gates such as AND, OR, NOT, XOR, and are widely used in arithmetic units, data processing systems, and control units. In our project, we focus on key combinational circuits including half adder, full adder, half subtractor, full subtractor, and decoder, each of which serves an essential function in digital computation.

A half adder is a simple circuit that adds two single-bit binary numbers and generates a sum and a carry. A full adder extends this by including a carry input, allowing multi-bit binary addition. Similarly, a half subtractor performs binary subtraction of two bits and produces a difference and a borrow, while the full subtractor handles subtraction involving a borrow input. The decoder is another important circuit that translates binary input codes into a corresponding one-hot output, which is commonly used for memory addressing and control logic.

However, like any hardware system, combinational circuits are susceptible to faults due to manufacturing defects, aging, or environmental damage. Fault detection is a technique used to identify such issues to ensure proper functionality of the circuit. The process involves generating specific input combinations (test vectors), simulating both fault-free and faulty scenarios, and comparing the outputs. Common faults include stuck-at-0 or stuck-at-1, where a line in the circuit is fixed at a constant value regardless of input. By detecting these faults early, the reliability, safety, and performance of digital systems can be greatly improved.

Merits and Demerits

Merits

Fault detection in combinational circuits is relatively straightforward due to the absence of memory elements, as the output depends solely on the current input values. This makes it easier to analyze and predict circuit behavior under different input conditions. Techniques such as truth table analysis, Boolean expression simplification, and simulation tools allow quick identification of faults like stuck-at-0, stuck-at-1, or open circuits. Additionally, automatic test pattern generation (ATPG) is more efficient in combinational circuits, enabling faster testing and debugging. The simplicity of these circuits also reduces testing time and cost, making fault detection more efficient and reliable, especially in smaller or less complex designs.

Demerits

Despite its simplicity, fault detection in combinational circuits has several limitations. As the complexity of the circuit increases, the number of possible input combinations grows exponentially, making exhaustive testing time-consuming and impractical. Traditional fault detection methods may only identify single faults, while multiple or intermittent faults can remain undetected. Additionally, these methods often fail to detect timing-related issues such as glitches or hazards, which can still affect the circuit's performance. Combinational fault detection also does not account for faults that may arise from interactions with sequential components or external environmental factors, limiting its effectiveness in real-world applications.

Aim:

This project aim is to develop a fault detection mechanism for combinational circuits (Adders, Subtractors and Decoder) to enhance their reliability and accuracy in digital systems.

Objectives:

The objective is to understand the types of faults commonly found in combinational logic circuits and how they affect circuit behavior. The project focuses on applying fault models like stuck-at faults and generating test vectors to detect these issues. It can detect single fault only. Through this, the project ensures early identification of faults and improves the overall performance of digital systems

Applications:

Fault detection in combinational circuits plays a crucial role in ensuring the reliability and proper functioning of digital systems. It is widely used in manufacturing testing of integrated circuits (ICs) to identify defects before the products are shipped. In digital communication systems, fault detection ensures accurate signal processing by verifying the integrity of logic circuits. It is also essential in embedded systems and consumer electronics, where consistent performance is critical. Additionally, fault detection is applied in automotive electronics, such as control units and sensors, to prevent system failures. In aerospace and defense, it ensures safety by detecting faults in mission-critical hardware. Overall, it enhances system dependability, reduces maintenance costs, and supports the design of more robust digital systems.

CHAPTER-2

FAULT DETECTION IN COMBINATIONAL CIRCUIT

Basic Components:

Bread Board:

A breadboard, solderless breadboard, or protoboard is a construction base used to build semi-permanent [prototypes](#) of [electronic circuits](#). Unlike a [perfboard](#) or [stripboard](#), breadboards do not require [soldering](#) or destruction of tracks and are hence reusable. For this reason, breadboards are also popular with students and in technological education.

A variety of electronic systems may be prototyped by using breadboards, from small analog and digital circuits to complete [central processing units](#) (CPUs).

Compared to more permanent circuit connection methods, modern breadboards have high [parasitic capacitance](#), relatively high resistance, and less reliable connections, which are subject to jostle and physical degradation. Signaling is limited to about 10 MHz, and even well below that frequency not everything works properly.

D Flip-Flop:

The D flip-flop (Data or Delay flip-flop) is a type of sequential circuit used to store a single bit of data. Unlike combinational circuits, it has a memory element and works based on a clock signal. When the clock edge (usually rising or falling) occurs, the value at the input 'D' is transferred to the output 'Q' and held there until the next clock edge. This makes the D flip-flop useful for data storage, synchronization, and timing applications. In digital systems, it is widely used in registers, counters, and memory circuits. While our project mainly focuses on combinational circuits, understanding D flip-flops provides insight into how data is temporarily stored and managed in more complex digital systems.

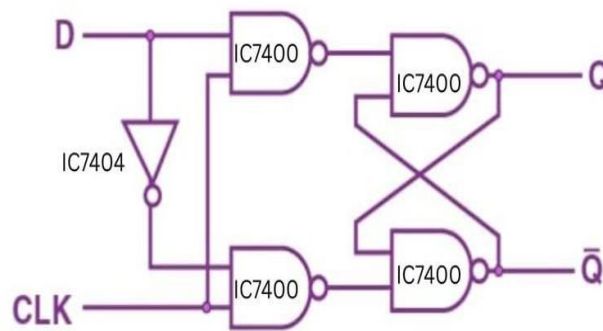


Fig 2.1: D Flip-Flop

Truth Table

Q	D	$Q_{(t+1)}$
0	0	0
0	1	1
1	0	0
1	1	1

Fig 2.2: D Flip-Flop Truth Table

Logic Gates:

Logic in digital electronics refers to the set of rules and operations used to process binary values (0s and 1s). It forms the basis of all digital circuits and systems.

In our project, logic means using logical operations like AND, OR, NOT, etc., to control how inputs are combined to produce the correct output. This helps in checking and detecting any faults in the circuit behavior.

Logic gates are the building blocks of combinational circuits. They perform basic logical operations on one or more binary inputs to produce a binary output. Here are the main gates used:

AND Gate:

The AND gate is a fundamental logic gate used in digital circuits. It produces an output of 1 only when all its inputs are 1; otherwise, the output is 0. This gate performs logical multiplication and is widely used in decision-making operations within combinational circuits.

The commonly used IC number for an AND gate is:

IC 7408

It contains four independent 2-input AND gates.

It belongs to the TTL (Transistor-Transistor Logic) family.

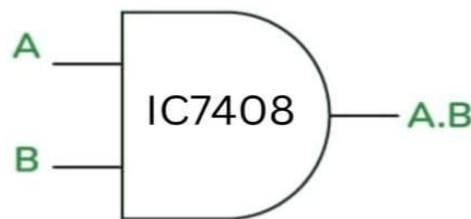


Fig 2.3: AND Gate

Truth Table

A (Input 1)	B (Input 2)	X = (A.B)
0	0	0
0	1	0
1	0	0
1	1	1

Fig 2.4: AND Gate Truth Table

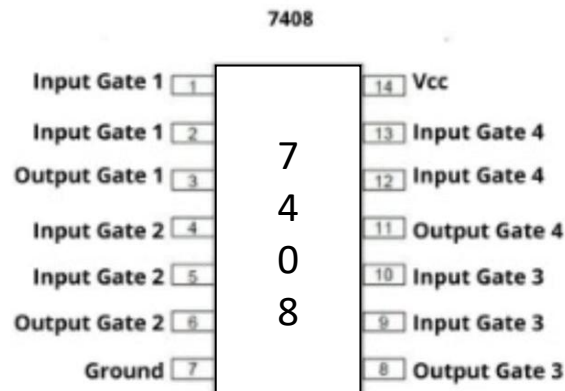


Fig 2.5: Pin Diagram

OR Gate:

The OR gate is a basic logic gate that gives an output of 1 if at least one input is 1. The output is 0 only when all inputs are 0. It performs logical addition and is used in circuits where any one condition being true should produce a high output.

The commonly used IC for OR gate is IC 7432.

It contains four 2-input OR gates.

It is part of the TTL logic family.



Fig 2.6: OR Gate

Truth Table

Input A	Input B	Output
0	0	0
0	1	1
1	0	1
1	1	1

Fig 2.7: OR Gate Truth Table

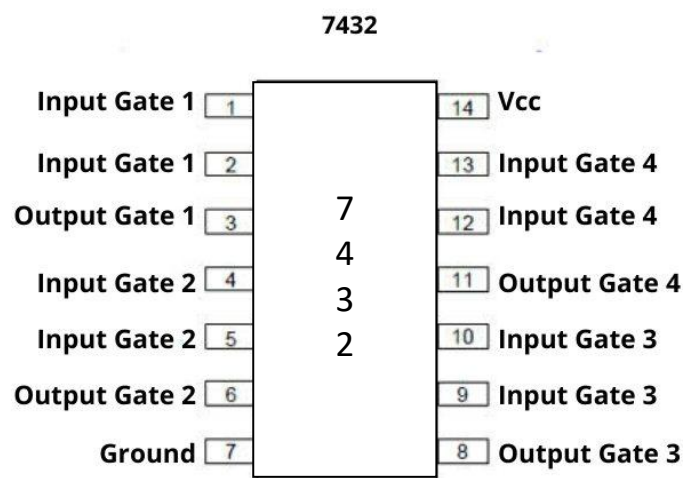


Fig 2.8: Pin Diagram

XOR Gate (Exclusive OR):

The XOR gate gives a high output (1) only when the inputs are different. If both inputs are the same (0,0 or 1,1), the output is 0. It is useful in circuits where the output depends on the difference between inputs, such as in fault detection or arithmetic operations.

The commonly used IC for XOR gate is IC 7486.

It contains four 2-input XOR gates.

Belongs to the TTL family of logic ICs.

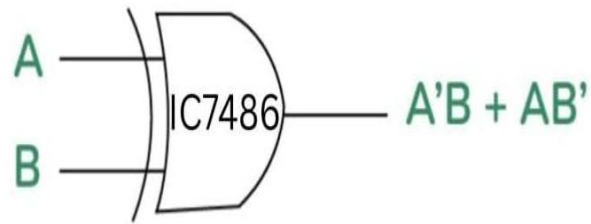


Fig 2.9: XOR Gate

Truth Table

A (Input 1)	B (Input 2)	X = A'B + AB'
0	0	0
0	1	1
1	0	1
1	1	0

Fig 2.10: XOR Gate Truth Table

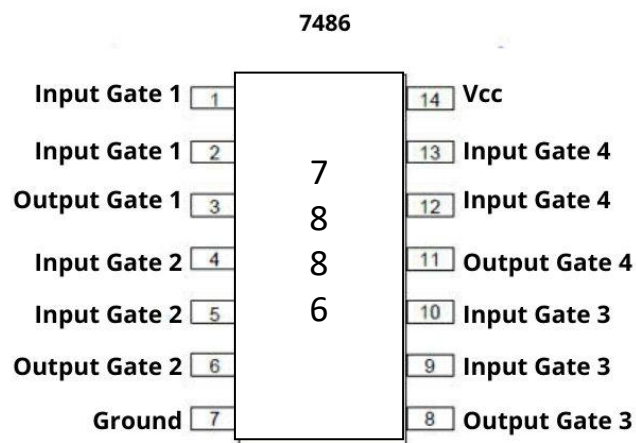


Fig 2.11: pin diagram

NOT Gate (Inverter):

The NOT gate is a logic gate that inverts the input. If the input is 1, the output is 0, and if the input is 0, the output is 1. It performs logical negation and is used to reverse the logic state in a circuit.

The commonly used IC for NOT gate is IC 7404.

It contains six independent NOT gates (inverters).

Part of the TTL logic family.



Fig 2.12: NOT Gate

Truth Table	
A (Input)	$Y = \bar{A}$ (Output)
0	1
1	0

Fig 2.13: NOT Gate Truth Table

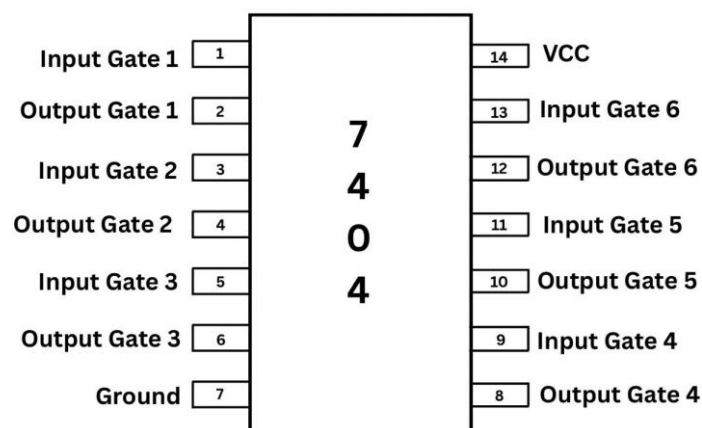


Fig 2.14: Pin Diagram

LED (Light Emitting Diode):

A Light Emitting Diode (LED) is a special type of diode that emits light when an electric current passes through it in the forward direction. It is made from semiconductor materials, and it works on the principle of electroluminescence—where energy is released in the form of light when electrons and holes recombine. LEDs are highly efficient compared to traditional light sources because they do not produce much heat and consume very little power. They are available in various colors, such as red, green, blue, yellow, and white, depending on the materials used during manufacturing. LEDs are commonly used in electronic circuits as indicators to show the status of devices, like whether a system is on or off, or to indicate faults in digital logic circuits. In practical use, a resistor is always connected in series with an LED to prevent too much current from flowing through it, which could damage the component. Due to their long life, low energy consumption, compact size, and fast switching ability, LEDs are widely used in display systems, flashlights, remote controls, medical devices, and decorative lighting.

Regulated Power Supply:

A Regulated Power Supply (RPS) is an essential electronic circuit or device that provides a constant and stable output voltage, regardless of variations in input voltage or load conditions. It is used to power sensitive electronic components and systems that require a specific and consistent voltage level to operate correctly. The main components of a regulated power supply include a transformer, which steps down the AC voltage; a rectifier, which converts AC to DC; a filter, which smooths out the ripples; and a voltage regulator, which maintains a constant output voltage. The regulator ensures that even if the input voltage fluctuates or the load changes, the output remains stable, protecting the connected components from damage. Regulated power supplies are commonly used in laboratories, communication equipment, embedded systems, and any circuit that requires reliable power. They are available in fixed and variable voltage types and can be designed for linear or switching operations depending on the application's efficiency and power requirements.

Jumper Wires: Jumper wires are short, flexible wires used to connect components on a breadboard or in circuits without soldering. They come in different types—male-to-male, male-to-female, and female-to-female—and are useful for testing and building temporary electronic setups. In digital projects, they connect ICs, LEDs, and other parts, making circuit design quick and easy.

CHAPTER-3

WORKING PRINCIPLE

The working principle of this project is based on the detection of faults in combinational logic circuits using a Finite State Machine (FSM)–driven approach. The primary goal is to identify and detect single stuck-at faults, namely stuck-at-0 (SA0) and stuck-at-1 (SA1), which are among the most common faults in digital logic. These faults occur when a signal line or node in the circuit becomes permanently stuck at a logic ‘0’ or logic ‘1’, regardless of the intended operation.

To detect these faults effectively, the process begins by constructing the truth table for combinational circuit and both its fault-free and fault-injected versions. Faults are manually introduced at different locations within the circuit—such as gate inputs, outputs, or internal interconnects—to simulate real-world conditions. Each fault is evaluated by applying every possible input combination to both versions of the circuit and recording the resulting outputs. By comparing the output of the fault-free circuit with that of the faulty circuit for each input combination, we can determine which inputs cause a deviation in the output behavior. These specific inputs are known as test vectors.

The selection of test vectors is a critical step. Only those input combinations that successfully detect at least one fault (i.e., cause a difference between fault-free and faulty outputs) are retained. Further analysis is performed to ensure that the chosen set of test vectors collectively covers all detectable single stuck-at faults at every fault location in the circuit. This optimization step reduces redundancy and limits the number of test vectors, improving the efficiency of the testing process while maintaining complete fault coverage.

Once the final set of test vectors is identified, a Finite State Machine (FSM) is designed to sequentially generate these test patterns. The FSM consists of a finite number of states, each corresponding to a specific test vector. Upon transitioning from one state to the next, the FSM produces a new input combination, which is then applied to the circuit under test. The FSM is implemented using counters.

As each test vector is applied, the corresponding output of the circuit is captured and compared with the known correct output (from the fault-free truth table). If the output deviates from the expected value for any test vector, a fault is detected.

CHAPTER-4

FAULT DETECTION

Fault Detection

Fault detection is the process of identifying errors or malfunctions in a digital circuit. In this project, fault detection is performed by applying test vectors to combinational logic circuits such as adders, subtractors, and decoders. Common faults like stuck-at-0 and stuck-at-1 are identified by comparing the actual outputs with the expected outputs. If the results do not match, a fault is detected. This method ensures the circuit functions correctly and reliably, even in the presence of faults.

Stuck-at-0 Fault

A stuck-at-0 fault occurs when a signal line or node in a digital circuit is permanently fixed at logic level 0, regardless of the actual input or logic condition. This fault prevents the line from going high (1) even when it should, leading to incorrect circuit behavior. In fault detection, test vectors are used to check if the circuit output changes correctly. If the output remains 0 when it should be 1, a stuck-at-0 fault is detected.

Stuck-at-1 Fault

A stuck-at-1 fault occurs when a signal line or node in a digital circuit is permanently fixed at logic level 1, regardless of its actual intended value. This fault causes the output to stay high (1) even when it should be low (0), leading to incorrect results in circuit operation. By applying specific test vectors and comparing the output with expected values, a stuck-at-1 fault can be detected when the output incorrectly remains at 1.

Adders:

Adders are fundamental combinational logic circuits used extensively in digital electronics and computer architecture to perform arithmetic operations, particularly binary addition. In binary systems, addition is a primary operation that forms the basis for more complex arithmetic functions such as subtraction, multiplication, and division. Adders are employed in devices like microprocessors, arithmetic logic units (ALUs), and calculators. Their main task is to compute the sum of binary numbers, and depending on the type, they can handle operations ranging from single-bit to multi-bit binary additions. These circuits work by

manipulating logical gates such as XOR, AND, and OR to simulate the arithmetic addition of binary values.

Types of Adders:

There are two basic types of binary adders:

1. Half Adder
2. Full Adder

Both serve the purpose of binary addition, but they differ in complexity and functionality. A Half Adder is a simple combinational circuit that adds two single-bit binary numbers and generates two outputs: Sum and Carry. It is called a "half" adder because it does not account for any carry input that might result from a previous stage of addition. In contrast, a Full Adder is more advanced and capable of adding three inputs: two significant binary digits and an input carry (C_{in}) from a previous addition. It also produces two outputs: a Sum and a Carry-out (C_{out}). For multi-bit binary numbers, multiple full adders can be connected in series to form ripple carry adders, allowing the system to perform addition of larger binary numbers by propagating the carry from one stage to the next.

Half Adder:

The Half Adder is the simplest form of an adder circuit. It accepts two binary inputs, labeled A and B, and produces two outputs — the Sum (S) and the Carry (C). The logic behind the half adder is that the sum is high (1) only when the inputs are different, and the carry is high (1) only when both inputs are high. This is implemented using basic logic gates: the XOR gate is used to generate the Sum, and the AND gate is used to generate the Carry.

The Boolean expressions for the outputs of a half adder are:

$$\text{Sum (S)} = A \oplus B$$

$$\text{Carry (C)} = A \cdot B$$

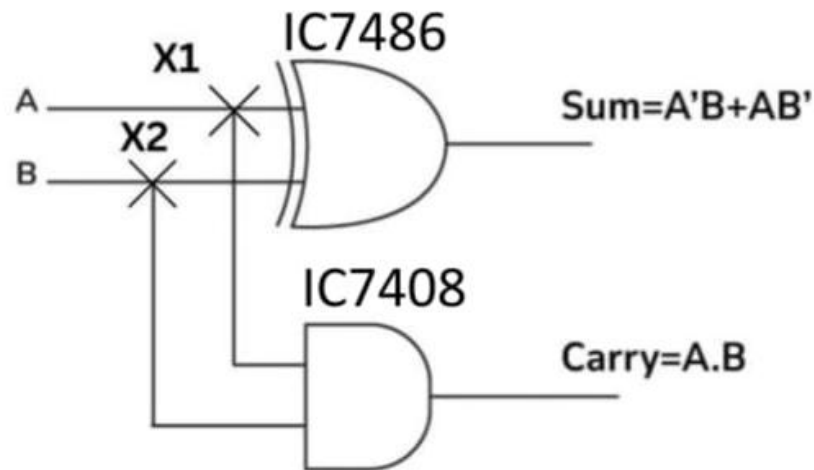


Fig 4.1: Half Adders Circuit

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Fig 4.2: Truth Table

This table clearly shows that a carry is generated only when both inputs are 1. However, since the half adder cannot accept a carry input from a previous stage, it is not sufficient for adding multi-bit binary numbers on its own. Therefore, it is typically used in simpler applications or as a building block within a full adder.

				STUCK AT 0			
A	B	SUM	CARRY	S	C	S	C
0	0	0	0	0	0	0	0
0	1	1	0	0	1	0	0
1	0	1	0	0	0	1	0
1	1	0	1	1	0	1	0

Fig 4.3: STUCK – AT 0 FAULT

				STUCK AT 1			
A	B	SUM	CARRY	S	C	S	C
0	0	0	0	0	1	0	1
0	1	1	0	0	0	1	1
1	0	1	0	1	0	0	1
1	1	0	1	0	1	0	1

Fig 4.4: STUCK – AT 1 FAULT

TEST VECTORS

To detect these faults, test vectors such as (0,0) and (1,1) are applied. These vectors are specific input combinations used to identify whether the circuit is functioning correctly or if a stuck-at fault is present. By comparing the expected outputs with the actual outputs under these test conditions, faults can be easily detected

(0,0) and (1,1)

PS (Present State)		NS (Next State)			
Q2	Q1	Q2+1	Q1+1	D1	D2
0	0	1	1	1	1
1	1	0	0	0	0

Fig 4.5: D – Flip Flop

K – MAP:

Q2 \ Q1	0	1
0	1	
1		

a) $D2 = Q_2 \cdot Q_1'$

Q2 \ Q1	0	1
0	1	
1		

b) $D1 = Q_1' \cdot Q_2'$

Fig 4.6: a) D1 and b) D2

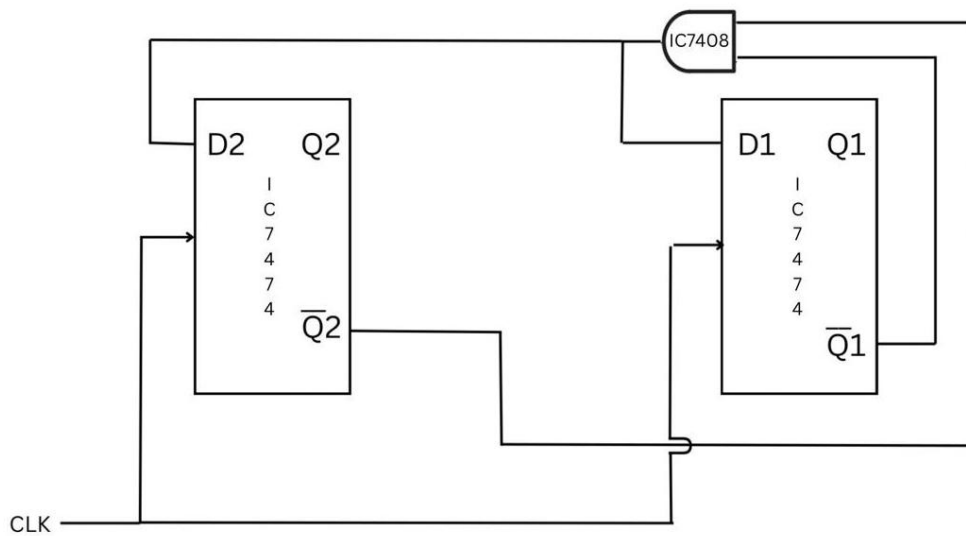


Fig 4.7: Counters

Full Adder:

The Full Adder circuit overcomes the limitation of the half adder by including a third input known as the carry-in (C_{in}). This allows the full adder to be used in cascading arrangements to build adders for multiple-bit binary numbers, where each adder handles a single bit and passes its carry to the next higher-order bit. The full adder has three inputs: A, B, and C_{in} , and two outputs: Sum (S) and Carry-out (C_{out}). The Sum output is the result of adding all three inputs, while the Carry-out is high when any two or more of the inputs are high, indicating that a carry must be forwarded to the next stage of addition.

The Boolean expressions for the outputs of a full adder are:

$$\text{Sum (S)} = A \oplus B \oplus C_{in}$$

$$\text{Carry-out (Cout)} = (A \cdot B) + (B \cdot C_{in}) + (A \cdot C_{in})$$

This equation for the carry can also be interpreted as the majority logic, meaning the carry output is high if at least two of the three inputs are high.

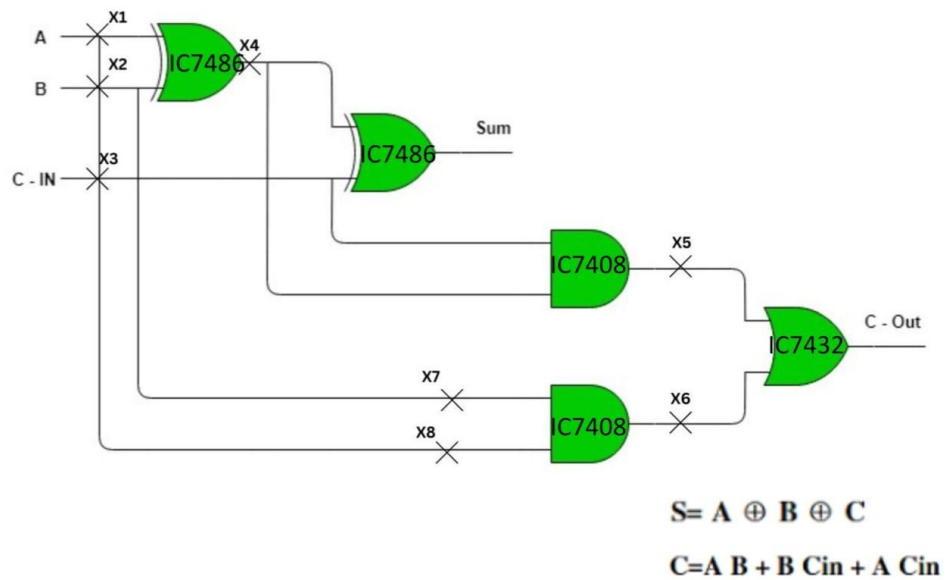


Fig 4.8: Full Adder Circuit

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Fig 4.9: Truth Table

This table clearly shows that a carry is generated in two cases: when both inputs A and B are 1, or when either A or B is 1 and there is a carry-in (Cin) of 1. This is because a full adder considers three inputs: A, B, and Cin. The sum output is 1 when the number of 1s among the three inputs is odd, and the carry output is 1 when at least two of the inputs are 1. Unlike the half adder, the full adder can accept a carry from a previous stage, making it suitable for constructing cascaded adders for multi-bit binary addition. It plays a crucial role in arithmetic circuits, enabling accurate and efficient binary addition in digital systems.

STUCK AT 0																				
A	B	Cin	S	Cy	S	X1 Cy	S	X2 Cy	S	X3 Cy	S	X4 Cy	S	X5 Cy	S	X6 Cy	S	X7 Cy	S	X8 Cy
0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
0	0	1	1	0	1	0	1	0	0	0	0	1	0	1	0	1	0	1	0	0
0	1	0	1	0	1	0	0	0	0	1	0	0	0	1	0	1	0	1	0	0
0	1	1	0	1	0	1	1	0	1	0	0	1	0	0	1	0	1	0	0	1
1	0	0	1	0	0	0	0	1	0	1	0	0	0	1	0	1	0	1	0	0
1	0	1	0	1	1	0	0	1	1	1	0	1	0	0	1	0	1	0	0	1
1	1	0	0	1	1	0	1	0	0	0	1	0	1	0	0	0	0	1	0	0
1	1	1	1	1	0	1	0	1	0	1	0	1	1	1	0	1	0	1	0	0

Fig 4.10: STUCK – AT 0 FAULT

STUCK AT 1																				
A	B	Cin	S	Cy	S	X1 Cy	S	X2 Cy	S	X3 Cy	S	X4 Cy	S	X5 Cy	S	X6 Cy	S	X7 Cy	S	X8 Cy
0	0	0	0	0	0	1	0	1	0	1	0	1	0	0	0	0	0	0	1	0
0	0	1	1	0	0	1	0	1	1	0	0	0	1	1	0	1	0	1	1	1
0	1	0	1	0	0	1	1	0	0	0	1	1	0	1	1	1	0	1	1	1
0	1	1	0	1	1	1	1	0	1	0	1	0	1	0	1	0	1	0	1	1
1	0	0	1	0	1	0	0	1	0	0	1	1	0	1	0	1	1	1	1	1
1	0	1	0	1	0	1	1	1	0	1	0	1	0	1	0	1	0	1	0	1
1	1	0	0	1	0	1	0	1	1	1	1	0	0	0	1	0	1	0	1	1
1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1

Fig 4.11: STUCK – AT 1 FAULT

TEST VECTORS:

To detect these faults, specific test vectors such as (0,1,0), (0,1,1), (1,0,0), and (1,1,1) are applied. These vectors represent different combinations of inputs (A, B, Cin) and help in exposing any fault by comparing the actual output with the expected output. If there is a mismatch, a fault is detected in the circuit. This method ensures accurate fault detection and verification in digital systems.

(0,1,0), (0,1,1), (1,0,0), (1,1,1)

PS (Present State)			NS (Next State)			D3	D2	D1
Q3	Q2	Q1	Q3+1	Q2+1	Q1+1			
0	1	0	0	1	1	0	1	1
0	1	1	1	0	0	1	0	0
1	0	0	1	1	1	1	1	1
1	1	1	0	1	0	0	1	0

Fig 4.12: D – Flip Flop

K – MAP:

Q2Q1					
Q3		00	01	11	10
	0				1
	1	1			

$$a) D1 = Q_1'(Q_2 * Q_3)$$

Q2Q1					
Q3		00	01	11	10
	0				1
	1	1		1	

$$b) D2 = Q_1'(Q_2 * Q_3) + Q_3 Q_2 Q_1$$

Q2Q1					
Q3		00	01	11	10
	0			1	
	1	1			

$$c) D3 = Q_3 Q_2' Q_1' + Q_3' Q_2 Q_1$$

Fig 4.13: a) D1 b) D2 and c) D3

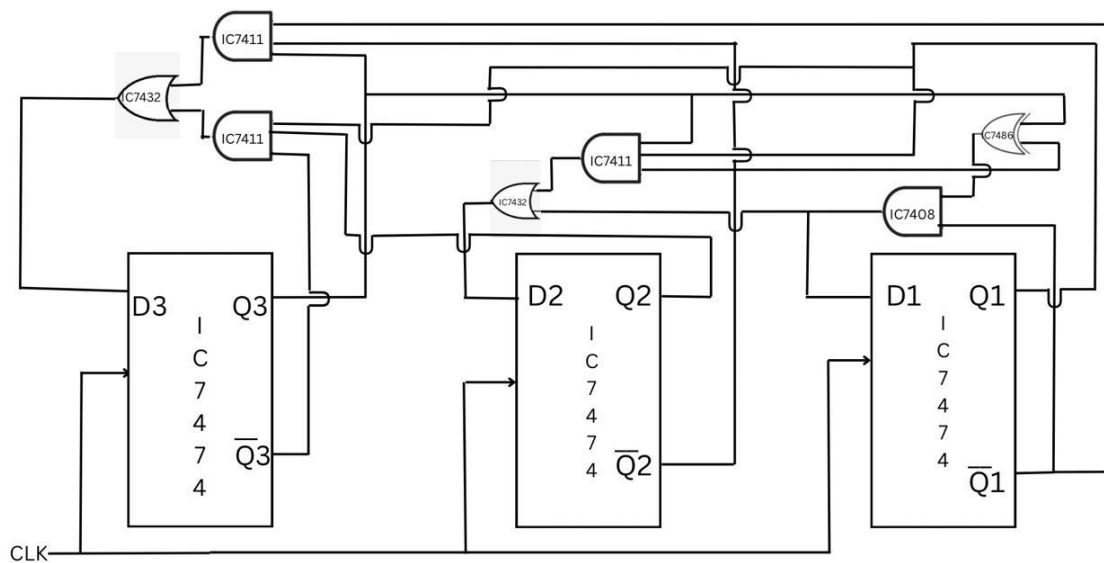


Fig 4.12: FSM to generate test vectors for full adder

SUBTRACTOR:

A Subtractor is a fundamental combinational logic circuit used to perform binary subtraction, just as an adder performs binary addition. In digital electronics, subtraction is essential in arithmetic operations, especially in applications like calculators, computers, and embedded systems. Subtractors take binary numbers as input and compute the difference and borrow (if needed). Like with addition, subtraction in binary also deals with two bits (or more), but instead of carry generation, borrow is generated whenever the subtrahend is larger than the minuend in a particular bit position.

Binary subtraction is based on four possible bit combinations:

$$0 - 0 = 0,$$

$$1 - 0 = 1,$$

$$1 - 1 = 0,$$

$$0 - 1 = 1 \text{ with borrow } 1.$$

From these basic rules, we build logical circuits to perform binary subtraction effectively.

Types of Subtractors

There are two basic types of subtractors used in digital logic design:

1. Half Subtractor
2. Full Subtractor

These subtractors are designed similarly to adders but are focused on performing subtraction logic and borrow handling. While a half subtractor works with two inputs (minuend and subtrahend), the full subtractor includes an additional input to account for a borrow-in, making it suitable for multi-bit subtraction when used in a cascade.

Half Subtractor

A Half Subtractor is a combinational logic circuit that subtracts one binary digit (B) from another (A) and provides the difference (D) and borrow (Bo) as outputs. It does not consider any previous borrow from a lower bit position, which means it is only suitable for the subtraction of the least significant bits or standalone 1-bit subtraction. The basic idea behind the half subtractor is to handle the four possible input combinations of A and B and determine when a borrow is needed. The circuit uses XOR and AND gates to implement its logic.

The Boolean equations for a half subtractor are:

$$\text{Difference (D)} = A \oplus B$$

$$\text{Borrow (Bo)} = A' \cdot B$$

Here, A' represents the NOT of A (i.e., complement of A). The XOR gate is used to find the difference, while the AND gate (with A inverted) is used to find when a borrow is required (only when A is 0 and B is 1).

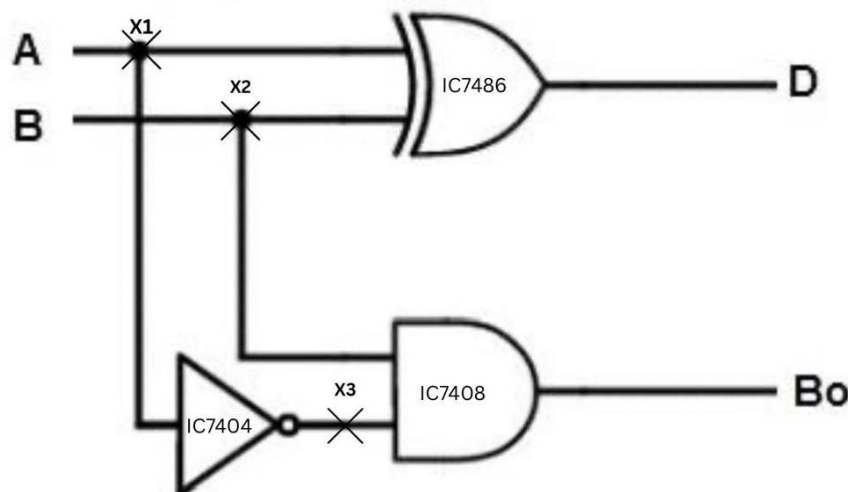


Fig 4.13: Half subtractor Circuit

Inputs		Outputs	
A	B	Difference	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Fig 4.14: Truth Table

As seen in the truth table, the borrow output is 1 only when A is 0 and B is 1 — this matches real binary subtraction behavior.

A	B	D (Diff)	B (Borrow)	STUCK AT 0					
				X1		X2		X3	
0	0	0	0	0	0	0	0	0	0
0	1	1	1	1	0	0	0	1	0
1	0	1	0	0	0	1	0	1	0
1	1	0	0	1	1	1	0	0	0

Fig 4.15: STUCK – AT 0 FAULT

A	B	D (Diff)	B (Borrow)	STUCK AT 1					
				X1		X2		X3	
0	0	0	0	1	0	1	1	0	0
0	1	1	1	0	0	1	1	1	1
1	0	1	0	1	0	0	0	1	0
1	1	0	0	0	0	0	0	0	1

Fig 4.16: STUCK – AT 1 FAULT

TEST VECTORS:

To detect these faults, test vectors such as (0,0), (0,1), and (1,1) are applied. These input combinations for A and B are selected to cover all logic possibilities of the subtractor. By comparing the actual output with the expected result for each vector, faults like incorrect borrow or difference values can be easily identified. This process ensures the accuracy and reliability of the subtractor circuit in digital systems.

(0,0), (0,1), (1,1)

PRESENT STATE		NEXT STATE			
Q2	Q1	Q2	Q1	D2	D1
0	0	0	1	0	1
0	1	1	1	1	1
1	1	0	0	0	0

Fig 4.17: D – Flip Flop

K – MAP:

Q2 \ Q1	0	1
0	1	1
1		

a) $D1=Q2'$

Q2 \ Q1	0	1
0		1
1		

b) $D2=Q2'Q1$

Fig 4.18: a) D1 b) D2

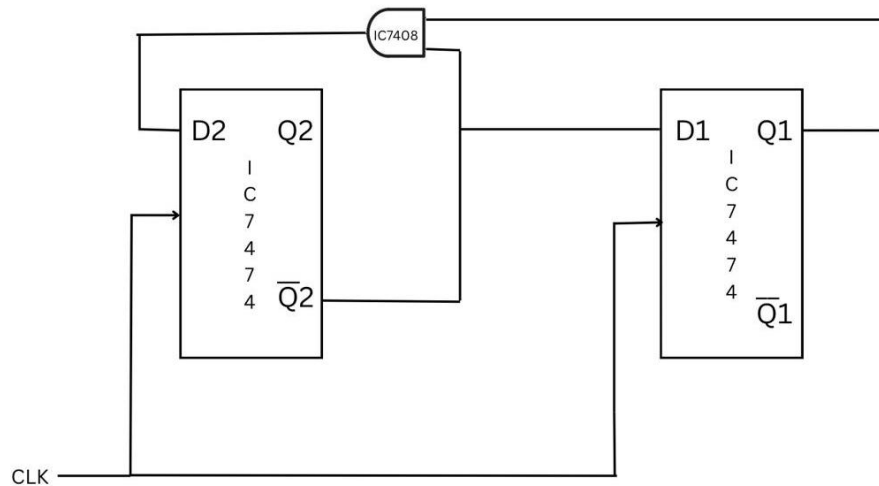


Fig 4.19: FSM to generate test vectors for half subtractor

Full Subtractor

The Full Subtractor is an extension of the half subtractor and is used to subtract three bits: the minuend (A), the subtrahend (B), and the borrow-in (Bin) from the previous lower stage. It produces two outputs: the difference (D) and the borrow-out (Bout). Full subtractors are essential when performing multi-bit binary subtraction (like 4-bit or 8-bit numbers), where the borrow from one bit affects the next bit's calculation. The design of the full subtractor involves combining two half subtractors and an OR gate to account for all input conditions.

The Boolean expressions for a full subtractor are:

$$\text{Difference (D)} = A \oplus B \oplus \text{Bin}$$

$$\text{Borrow-out (Bout)} = (A' \cdot B) + (\text{Bin} \cdot (A \oplus B)')$$

Let's break it down:

The difference uses three-input XOR logic, which flips the output based on the number of 1s.

The borrow-out becomes 1 in cases where either $A < B$ or a borrow-in is needed due to earlier subtraction. The logic ensures that if the first subtraction caused a deficit, the borrow is carried forward.

Fig 4.22: STUCK – AT 0 FAULT

STUCK AT 1																					
A	B	bin	D	B_out		X1		X2		X3		X4		X5		X6		X7		X8	
0	0	0	0	0	0	1	1	1	0	1	1	1	0	0	0	0	0	0	1	0	1
0	0	1	1	1	1	1	1	0	0	0	1	0	0	1	1	1	1	1	1	1	1
0	1	0	1	1	1	0	1	0	0	1	1	1	1	1	1	1	1	1	1	1	1
0	1	1	0	1	1	0	1	1	1	0	1	0	1	0	1	0	1	0	1	0	1
1	0	0	1	0	0	0	0	1	0	0	0	1	0	1	0	1	0	1	1	1	1
1	0	1	0	0	0	0	0	0	0	1	1	0	0	0	0	0	1	0	1	0	1
1	1	0	0	0	0	1	1	0	0	0	0	1	0	0	1	0	0	0	1	0	1
1	1	1	1	1	1	1	1	1	1	1	1	0	0	1	1	1	1	1	1	1	1

Fig 4.23: STUCK – AT 1 FAULTS

TEST VECTORS

To detect these faults, specific test vectors such as (0,0,1), (0,1,0), (1,0,1), and (1,1,0) are used. These combinations of inputs (A, B, Bin) help in identifying faults by comparing actual outputs against expected results. If there is a mismatch, it indicates a fault in the logic line. These test vectors are carefully chosen to cover all significant input conditions and ensure reliable fault detection across the subtractor circuit.

(0,0,1), (0,1,0), (1,0,1), (1,1,0)

PS (Present State)			NS (Next State)						
Q3	Q2	Q1	Q3	Q2	Q1	D3	D2	D1	
0	0	1	0	1	0	0	1	0	
0	1	0	1	0	1	1	0	1	
1	0	1	1	1	1	0	1	0	
1	1	0	0	0	0	1	0	1	

Fig 4.24: D – Flip Flop

K – MAP:

Q2Q1		00	01	11	10
Q3	0				1
	1				1

$$a) D1 = Q2Q1'$$

Q2Q1		00	01	11	10
Q3	0		1		
	1		1		

$$b) D2 = Q2'Q1$$

Q2Q1		00	01	11	10
Q3	0				1
	1		1		

$$c) D3 = Q3Q2'Q1 + Q3'Q2Q1'$$

Fig 4.25: a) D1, b) D2 and c) D3

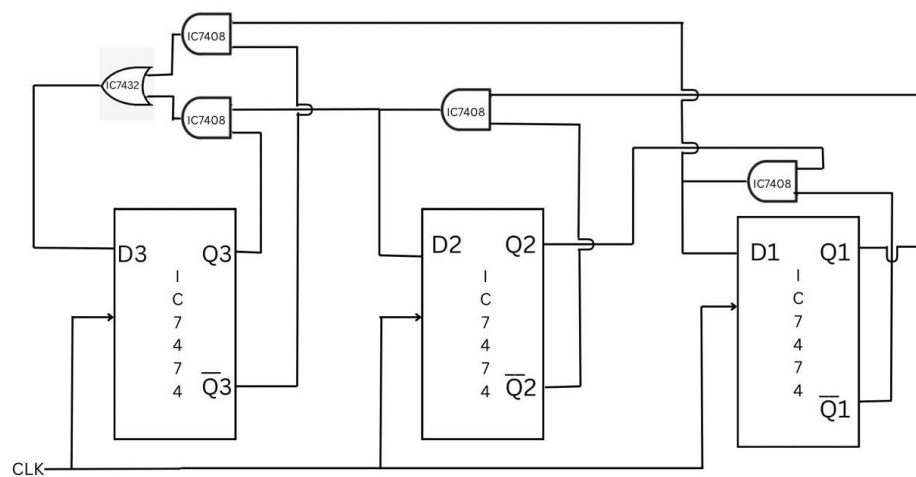


Fig 4.26: FSM to generate test vectors for full subtractor

Decoder

A decoder is a combinational digital circuit that takes a set of n input binary signals and activates one of 2^n output lines, depending on the combination of inputs. In simple words, a decoder converts binary-coded inputs into a unique active output line. The main function of a decoder is to identify or decode a particular binary code. For example, if you input 010 (which is decimal 2), the decoder will activate the third output line (counting from 0). All the other outputs will be set to 0. Decoders are widely used in various digital applications, including memory address decoding, data multiplexing, instruction decoding in CPUs, and display systems like 7-segment LED displays. They serve as building blocks in many digital systems, especially for selecting one out of many devices or memory locations.

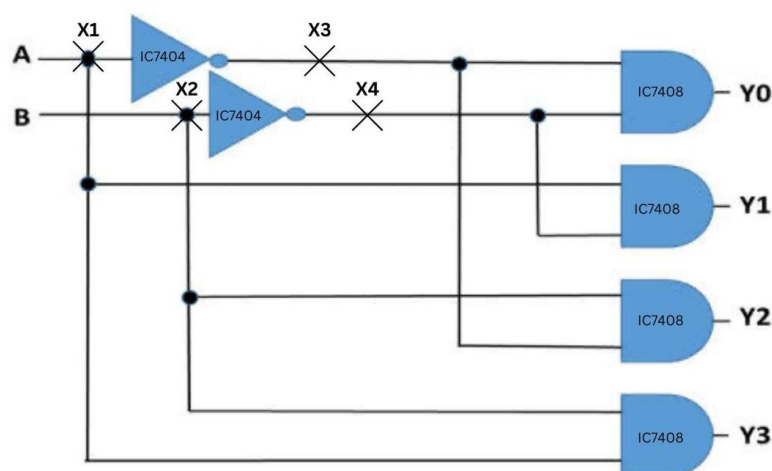


Fig 4.27: decoder Circuit

A	B	D0	D1	D2	D3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Fig 4.28: Truth Table

It takes two input variables, A and B, and produces four output lines: D0, D1, D2, and D3. For each combination of inputs, only one output line is set to 1 while the others remain 0.

This indicates that the decoder activates a single output corresponding to

the binary value of the inputs (00 activates D0, 01 activates D1, 10 activates D2, and 11 activates D3). This kind of decoder is commonly used in digital circuits for address decoding or data routing.

		STUCK AT 0																			
		X1				X2				X3				X4							
A	B	D0	D1	D2	D3	D0	D1	D2	D3	D0	D1	D2	D3	D0	D1	D2	D3	D0	D1	D2	D3
0	0	1	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
0	1	0	1	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0	1	0
1	0	0	0	1	0	1	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0
1	1	0	0	0	1	0	1	0	0	0	0	1	0	0	0	0	1	0	0	0	1

Fig 4.29: STUCK – AT 0 FAULT

		STUCK AT 1																			
		X1				X2				X3				X4							
A	B	D0	D1	D2	D3	D0	D1	D2	D3	D0	D1	D2	D3	D0	D1	D2	D3	D0	D1	D2	D3
0	0	1	0	0	0	0	0	1	0	0	1	0	0	1	0	0	0	1	0	0	0
0	1	0	1	0	0	0	0	0	1	0	1	0	0	1	1	0	0	0	1	0	0
1	0	0	0	1	0	0	0	1	0	0	0	0	1	0	0	1	0	1	0	1	0
1	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	1

Fig 4.30: STUCK – AT 0 FAULT

TEST VECTORS:

The test vectors (0,1) and (1,0) are applied to check whether each output responds correctly to the corresponding input combination. By observing deviations in the output lines (D0–D3), these test vectors help in identifying faulty lines and validating the decoder's performance under fault conditions.

(0,1) and (1,0)

PS (Present State)		NS (Next State)		D2		D1
Q2	Q1	Q2	Q1			
0	1	1	0	1	0	
1	0	0	1	0	1	

Fig 4.31: D – Flip Flop

K – MAP:

Q1 \ Q2	0	1
0		1
1		

a) $D2 = Q2'Q1$

Q1 \ Q2	0	1
0		
1	1	

b) $D1 = Q2Q1'$

Fig 4.32: a) D2 and b) D1

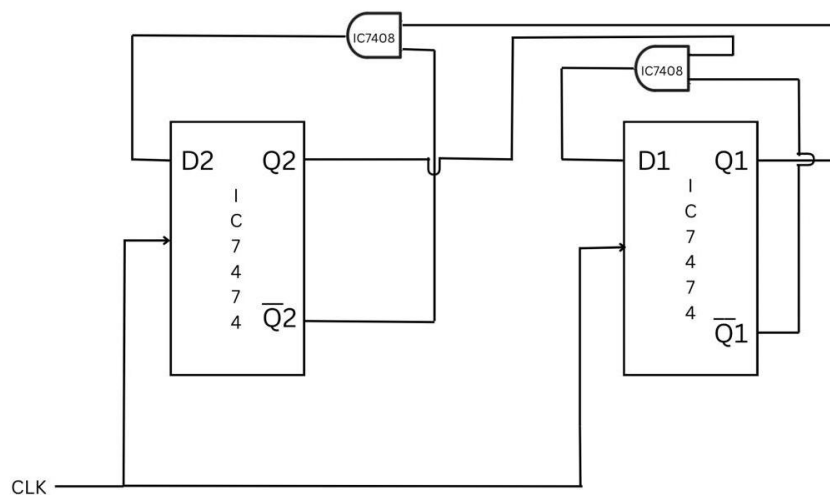


Fig 4.33: FSM to generate test vectors for decoder

CHAPTER-5

Conclusion and Future Scope

This project focused on identifying and diagnosing faults in fundamental combinational circuits such as adders, subtractors, and decoders. The main objective of this project is to reduce the input test vectors.

Through this work, we applied a systematic approach using truth tables, test vectors, and logic simulations to compare expected and actual outputs. We specifically targeted common faults like stuck-at faults (where a wire is stuck at logic 0 or 1), open circuits, and shorted outputs. Faults were identified when there was a mismatch between expected output and actual circuit behavior for specific input combinations.

Our project highlights the importance of early-stage testing and verification in digital design. By detecting faults early, we can reduce system failures, save repair costs, and improve the overall reliability of digital devices. The practical implementation of this approach can be extended into hardware testing environments and educational simulations.

This fault detection technique is particularly relevant in the fields of Embedded Systems, VLSI Design, Medical Devices, and Aerospace, where precision and fault-free operation are critical.

Future Scope

Integration with Fault Simulation Tools

In future implementations, simulation tools like ModelSim, Xilinx Vivado, or LTspice can be used to model and automatically detect faults. This will allow complex circuits to be tested more efficiently and with greater accuracy.

Fault Detection in Complex Circuits

The methodology used in this project can be extended to more complex circuits such as multipliers, dividers, multiplexers, ALUs, and even custom-designed digital logic systems.

Testing of Sequential Circuits

Sequential circuits like flip-flops, counters, and shift registers introduce memory elements. Fault detection in these systems is more complex but also more critical. Future work can focus on detecting faults that occur over time, such as clock glitches or data loss.

Real-Time Fault Monitoring Systems

Development of real-time fault detection systems using microcontrollers or FPGAs. These systems can continuously monitor circuit outputs and alert if any fault is detected.

Use of Artificial Intelligence & Machine Learning

With the advancement in AI/ML, intelligent systems can be trained on large sets of fault data to predict, classify, and even correct faults automatically without manual intervention.

Built-In Self-Test (BIST) Design

Modern circuits can include BIST logic, where the circuit tests itself periodically. Our project could lead into BIST circuit design for adders, subtractors, and decoders to enhance reliability in high-speed digital systems.

Low-Power and Fault-Tolerant Design

As digital systems become more compact and energy-efficient, designing fault-tolerant circuits that continue to function under partial failure is an important next step. Error-correcting techniques and redundancy could be explored.

Application in Safety-Critical Domains

This project lays the foundation for deploying these fault detection mechanisms in medical electronics, automotive electronics, and defense systems, where any fault can have serious consequences.

References

Defect and Fault Detection in Combinational Circuits: Techniques and Analysis

IEEE Conference Publication | IEEE Xplore

Test for Detection and Location of Intermittent Faults in Combinational Circuits

IEEE Journals & Magazine | IEEE Xplore

Fault Masking in Combinational Logic Circuits

IEEE Journals & Magazine | IEEE Xplore

Multiple Fault Detection in Combinational Circuits: Algorithm Computational Results

IEEE Journals & Magazine | IEEE Xplore